THE CULPRIT

AI Agents & Docs

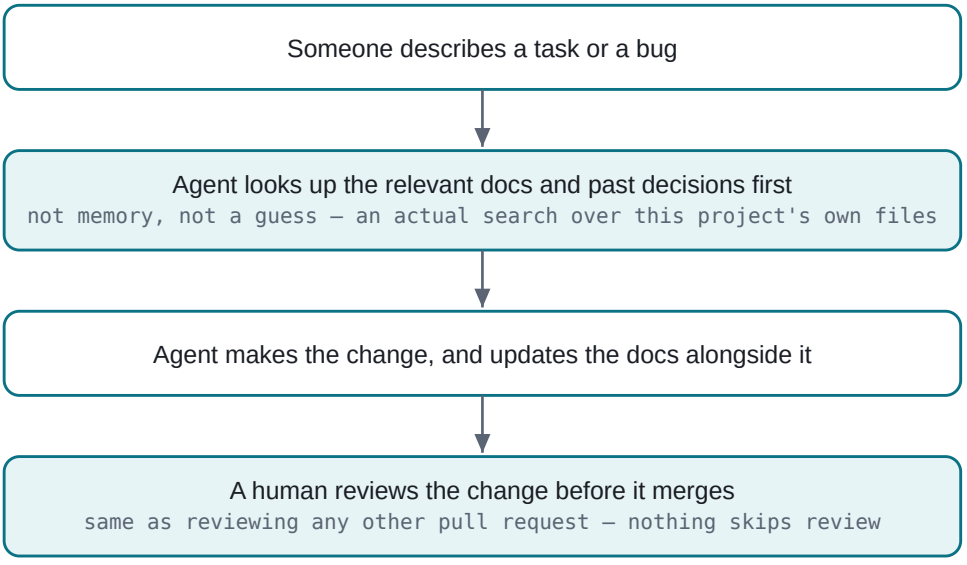
A plain explanation of how an AI coding agent gets the context it needs on this project, and where a human still has to step in. Written for handing to a team, not for debugging code.

`Retrieval: keyword search over the project's own docs`

`No external AI service is called to do the searching` `Reviewed: 13 Aug 2026`

01 How a Change Happens

An AI agent never edits this project from a blank slate. Before touching any code, it's expected to go read what's already been decided — the same way a new hire would be expected to read the project handbook before their first commit, not guess.



The loop is deliberately boring: look things up, make the change, keep the docs honest, let a human sign off. Nothing about it skips the normal review a human contributor's change would also go through.

02 How an Agent Finds the Right Answer

The project has a small built-in search tool (`npm run docs:search -- "some question"`) that both AI agents and human developers use to find relevant project documentation before making a change. It's worth being precise about what kind of tool this actually is, because the honest answer is simpler — and more inspectable — than it might sound.

It's a keyword search, not a call to an external AI service. It reads every Markdown doc in the project, splits each one into sections by heading, and scores each section against the question: does the section's title match? Does the exact phrase appear? How often do the question's words show up? Nothing here sends project data anywhere — it all runs locally, in plain code anyone on the team can read top to bottom.

Two small judgment calls make the results noticeably more useful than a plain word match would be:

- **Current beats outdated, unless you're explicitly asking about history.** Every doc is tagged `current` , `deprecated` , or `historical` . A current doc ranks higher by default, so an agent asking "how does login work" doesn't surface a removed approach by accident. Asking a history-flavored question ("why was X replaced") flips that weighting on purpose.
- **A decision record ranks higher than a casual mention.** The project keeps one dated record per significant decision (an "ADR" — the reasoning for *why* something was built a particular way, alternatives considered included). Those records are weighted up, since "why" is usually what's actually being asked for.

This is a lighter-weight stand-in for what's often called a RAG pipeline — retrieval before generation — just built from plain keyword scoring instead of an embedding/vector database. The tool was written so that swapping in a smarter, semantic version later wouldn't require changing how an agent calls it — but there was no need to reach for that complexity yet on a project this size.

03 The Knowledge Base

What the search tool actually searches is a small, curated set of files — not the whole codebase, and not the whole internet.

LAYER	WHAT IT IS	KEPT HONEST BY
Decision records	One dated write-up per significant decision: what was decided, what else was considered, and why. The authoritative record of *why*, not just *what*.	Never edited after the fact — a reversed decision gets a *new* record, not a rewrite of the old one
Current-state docs	Short reference pages — architecture, requirements, deployment — describing the project as it actually is today.	Cross-checked against the real code, not just against what was originally planned
This published site	The human-facing rendering of the above — what you're reading right now.	Regenerated from the same source files, not maintained separately by hand

The rule that keeps an agent from confidently resurrecting something the team already decided against: nothing is silently deleted when it changes. A removed feature gets a labeled note pointing at the decision record that removed it, so both a human skimming the docs and an agent searching them see the same "this was tried, and here's why it isn't there anymore" — instead of the removal just quietly vanishing from the record.

04 What's Shared, What Stays Private

Not everything an agent reads is meant for a wide audience, and that's an intentional line, not an oversight.

KEPT PRIVATE	SHARED WITH THE TEAM / PUBLIC
Raw planning notes and the full narrative history behind the project	The distilled, current-state version of the same facts, cross-checked against the real code
Internal AI-agent configuration (how an agent is instructed to behave)	The plain-language contract for what an agent is expected to do before and after a change (read first, update docs after, never skip review)
Raw meeting/requirement notes as originally given	The decision records that explain what was ultimately built, and why, in finished form

The pattern is consistent throughout: working notes stay private, and the *conclusions* drawn from them are what gets published, reviewed, and kept up to date as a durable public record.

05 The Human's Part

None of the above replaces a human's judgment — it just means the human's attention goes to the decisions that actually need it, instead of to re-deriving context an agent could look up itself. Concretely, as the person connecting the team to the agents working on it, what's worth actually checking:

- **Did a real decision get a decision record?** A change that quietly reverses or replaces something earlier, without one, is the one pattern worth catching in review — it's exactly the gap that would make the next lookup wrong.
- **Did the docs move with the code, or trail behind it?** A change that touches how the project works but leaves the docs describing the old way is worse than no docs at all, because it looks trustworthy while being wrong.
- **Is anything marked "current" actually still current?** That label is a claim, not a guarantee — it's only as reliable as the last person (or agent) who checked it.

> In short: the retrieval tool and the knowledge base mean an agent starts from the same shared understanding a briefed human would, instead of guessing. What still needs a human is judging whether that shared understanding is actually being kept straight over time.